

POA v1.1 — Relatório de Mudanças Implementadas

Projeto: POA — Pipeline de Otimização de Antígenos **Branch:** `feature/viz-realdata` (derivada de `feature/automation-gui`, base `main`) **Data:** 17 de agosto de 2026 **Base de comparação:** commit `62cf8f4` (`main`)

1. Resumo executivo

O POA era um pipeline semiautomático composto por scripts Python soltos na raiz do repositório, com todas as submissões às ferramentas web feitas manualmente pelo usuário. Esta entrega reorganiza o código em um pacote Python em camadas, adiciona uma camada de automação para as ferramentas externas, uma interface gráfica web local, visualização 2D/3D dos epítomos e uma suíte de testes — **sem alterar a lógica científica de ranqueamento e seleção de epítomos** e **sem quebrar a linha de comando original**.

Indicador	Antes (<code>main</code>)	Depois (<code>feature/viz-realdata</code>)
Organização do código	13 scripts na raiz	pacote <code>poa/</code> em 4 camadas (44 arquivos)
Testes automatizados	nenhum	54 testes (<code>pytest</code>), todos passando
Interface	apenas linha de comando	CLI (inalterada) + GUI Streamlit de 7 etapas
Submissão às ferramentas web	100% manual	automatizada ou reimplementada, com fallback manual
Análise de conservação	manual no site do IEDB	reimplementação local + upload manual
Visualização dos resultados	planilhas e FASTAs	mapa 2D de epítomos + estrutura 3D interativa
Documentação	README	README + <code>ARCHITECTURE.md</code> + <code>TESTING_WSL.md</code> + este relatório

Total consolidado do diff `main` → `feature/viz-realdata`: **63 arquivos alterados, 4.934 linhas adicionadas, 1.326 removidas** (as remoções são os scripts antigos migrados para o pacote).

2. Reorganização do código em pacote (`refactor`)

Os scripts da raiz foram convertidos em um pacote com separação clara de responsabilidades. Os arquivos originais `POA1_v1.0.py` e `POA2_v1.0.py` permaneceram como pontos de entrada finos (*shims*) que apenas chamam o pacote, de modo que todos os comandos documentados no README continuam funcionando exatamente como antes.

Camada	Pacote	Responsabilidade
Núcleo	<code>poa/core</code>	Parsing dos formatos de predição, ranqueamento, relatório, topologia. Sem rede, sem I/O externo.
Serviços	<code>poa/services</code>	Comunicação com ferramentas externas (API, subprocesso, navegador), cache e fallback manual.
Linha de comando	<code>poa/cli</code>	<code>poa1</code> , <code>poa2</code> e o novo <code>import_realdata</code> . Mesmos argumentos de antes.
Interface	<code>poa/gui</code>	Aplicação Streamlit e helpers de visualização.

Mapeamento dos scripts antigos:

Script original (removido)	Novo local
<code>PrEpiAn.py</code>	<code>poa/core/pipeline.py</code> (<code>consolidate_predictions</code>)
<code>bepipred.py</code>	<code>poa/core/parsers/bepipred.py</code>
<code>papImed.py</code>	<code>poa/core/parsers/papimed.py</code>
<code>netctl.py</code>	<code>poa/core/parsers/netctl.py</code>
<code>mhcii.py</code>	<code>poa/core/parsers/mhcii.py</code>
<code>othersPred.py</code>	<code>poa/core/parsers/others.py</code>
<code>conservancyAnalysis.py</code>	<code>poa/core/parsers/conservancy.py</code>
<code>TMHMM.py</code> , <code>PepiTMHMM.py</code>	<code>poa/core/topology.py</code>
<code>prepianResultsforConservancyAnalysis.py</code>	<code>poa/core/fasta_out.py</code>

Além disso, as chamadas `print` de diagnóstico foram substituídas por *logging* configurável (`poa/logging_conf.py`), e as colunas padronizadas do pipeline foram centralizadas em `poa/core/models.py` .

3. Camada de automação das ferramentas externas

Cada ferramenta externa ganhou um cliente próprio em `poa/services` . Todos seguem o mesmo contrato: quando o serviço não está disponível, lançam `ServiceUnavailable` , o pipeline registra uma mensagem clara e oferece o **upload manual do arquivo de resultado** — preservando integralmente o comportamento semiautomático original.

Ferramenta	Estratégia implementada	Fallback
MHC-II Binding (IEDB)	API REST legada síncrona (<code>tools-cluster-interface.iedb.org/tools_api/mhcii/</code>)	upload manual
BepiPred-3.0	pacote local <code>bp3</code> (PyTorch + ESM-2), saída FASTA já compatível com o parser	upload manual
PAP/IMED	EMBOSS <code>antigenic</code> local via subprocesso (mesmo método Kolaskar–Tongaonkar do site, que saiu do ar)	upload manual
Epitope Conservancy (IEDB)	reimplementação local do algoritmo	upload manual do CSV
NetCTL 1.2 / BepiPred-2.0	automação de navegador (Playwright), <i>best effort</i>	upload manual (padrão na GUI)
pyTMHMM	execução local, inalterada	—

Pontos de projeto relevantes:

- **Saída normalizada.** Cada cliente grava o resultado no mesmo formato que o parser correspondente já consumia. O cliente MHC-II, por exemplo, converte o TSV da API para o layout `<Proteína>_<Espécie>.html`, e o cliente EMBOSS converte o GFF para o layout `.txt` do PAP/IMED. Isso significa que **o caminho automático e o caminho manual usam exatamente o mesmo parser** — não há divergência de comportamento entre eles.
- **Cache em disco** (`poa/services/cache.py`): chave SHA-256 derivada de ferramenta, versão, parâmetros e conteúdo de entrada. Evita ressubmeter as mesmas sequências. Os arquivos ficam em `.poa_cache/<ferramenta>/<chave>.<ext>`, legíveis e fáceis de limpar.
- **Automação de navegador marcada como experimental.** NetCTL 1.2 e BepiPred-2.0 (DTU) só expõem formulários web assíncronos (submeter → id do job → *polling* → resultado), cujo HTML pode mudar sem aviso. Por isso a GUI mantém essas duas ferramentas em modo manual por padrão.

3.1 Reimplementação local da Epitope Conservancy Analysis

Verificamos que o IEDB **não oferece API nem versão standalone** para essa ferramenta, o que a tornava o principal gargalo manual entre POA1 e POA2. Com aprovação explícita do usuário — por tocar em lógica científica —, o algoritmo foi reimplementado localmente em `poa/services/conservancy_client.py`, seguindo Bui et al. (BMC Bioinformatics, 2007):

Para um epítipo linear de comprimento L , desliza-se uma janela sem *gaps* de comprimento L ao longo de cada proteína, calculando a identidade percentual (correspondências / $L \times 100$) em cada deslocamento; a identidade do epítipo com aquela proteína é o **máximo** entre todos os deslocamentos. Dado um limiar de identidade T , reporta-se por epítipo o percentual (e a contagem) de proteínas com identidade $\geq T$, além da identidade mínima e máxima observadas.

O CSV gerado usa exatamente os mesmos cabeçalhos de coluna que o parser do POA2 já consumia, portanto **o POA2 funciona de forma idêntica** com CSVs vindos do IEDB ou da reimplementação.

Recomendação de validação: antes de usar em produção, compare a saída local com uma exportação real do IEDB para o seu conjunto de dados. As verificações de equivalência estão em `tests/test_conservancy_local.py`.

4. Interface gráfica (Streamlit)

`poa/gui/app.py` implementa um fluxo guiado que orquestra o pipeline inteiro em **7 etapas**:

1. **Proteínas** — upload do FASTA de referência (`-f`).
2. **Predições** — dispara cada ferramenta automatizada ou aceita upload manual do resultado.
3. **POA1** — consolidação, ranqueamento e geração dos FASTAs por espécie.
4. **Conservancy Analysis** — execução local ou upload dos CSVs do IEDB.
5. **POA2** — filtro por conservação e classificação de topologia de membrana.
6. **Resultados** — tabelas, estatísticas e download dos `.xlsx` / `.fasta`.
7. **Visualização (2D/3D)** — mapa de epítomos e estrutura tridimensional (ver seção 5).

A orquestração ficou em `poa/gui/backend.py`, propositalmente **sem dependência do Streamlit**, o que permite testá-la sem subir a interface (`tests/test_gui_backend.py`).

Execução:

```
streamlit run poa/gui/app.py      # ou:  python run_gui.py
```

5. Visualização 2D e 3D dos epítomos (*novo nesta branch*)

Nova etapa da GUI, apoiada em `poa/gui/viz.py` (também livre de imports do Streamlit, portanto testável — `tests/test_viz.py`).

Mapa 2D de epítomos (Plotly). Cada epítomo é desenhado como um segmento horizontal sobre o eixo de resíduos, com uma trilha por `Proteína_Espécie` e cor por método de predição. Permite ver de relance onde os métodos concordam, quais regiões concentram epítomos e qual a extensão de cada um. Não exige estrutura tridimensional — funciona assim que o POA1 roda.

Visualizador 3D (py3Dmol / 3Dmol.js). O usuário envia um arquivo `.pdb` e escolhe de qual `Proteína_Espécie` destacar os epítomos; eles são realçados em vermelho sobre a estrutura, em representação *cartoon*, *stick* ou *sphere*, com superfície de van der Waals opcional — no estilo de visualização do Discovery Studio.

Duas ressalvas foram tratadas explicitamente na interface:

- **Numeração de resíduos.** A numeração do PDB precisa corresponder à posição na sequência usada nas predições. A interface exibe esse alerta junto da renderização, já que *gaps* ou deslocamentos de numeração produziriam destaques incorretos sem nenhum erro aparente.

- **Dependência de rede.** O 3Dmol.js é carregado de um CDN, então a visualização 3D exige internet no momento da renderização. Falhas são exibidas com `st.error`, sem derrubar a aplicação.

Nova dependência: `py3Dmol>=2.0` (adicionada ao `requirements.txt`).

6. Adaptador para os arquivos originais da pesquisa (*novo nesta branch*)

Ao testar o pipeline com os arquivos reais da pesquisa (`2_fastas_mundo/` , `5_predicao_de_epitopo/`), constatamos que eles **não seguem as convenções de cabeçalho** que os parsers esperam. O problema central: as exportações JSON do BepiPred-2.0 trazem uma única chave de antígeno genérica, `"Sequence"`. O parser extrai proteína/espécie/ID dessa chave via expressão regular `(\w+?)(\w+?)(\w+)`, de modo que uma chave genérica produzia `NaN` — e os arquivos de conservação saíam nomeados `nan_epitopes.fasta`.

`poa/services/realdata_import.py` resolve isso adaptando **apenas a nomenclatura, nunca os dados preditos**:

- Reescreve a chave do antígeno para o formato `Proteína_Espécie_ID` (ex.: `E_DENV1_ref`), copiando os escores por resíduo literalmente.
- Reconstrói o FASTA de referência (`-f`) a partir do próprio array `AA` do JSON — ou seja, a sequência exata sobre a qual o BepiPred rodou, eliminando qualquer risco de descompasso entre a referência e a predição.
- `best_world_match` mapeia automaticamente cada vírus ao seu conjunto de diversidade para a etapa de conservação, por identidade com o primeiro registro. Empates são desfeitos pelo comprimento mais próximo, o que faz o CHIKV escolher o conjunto de E1 em vez da poliproteína E2-E1-6K que meramente o contém.
- Referências que contêm o resíduo ambíguo `X` são sinalizadas com `has_x`, pois a checagem de integridade do POA1 as recusa como `-f`.

Validação cruzada: o epítipo `IGNRDFVE` (posições 6–13) reconstruído para DENV1 coincide exatamente com o `bepipred_denv1.fasta` produzido manualmente pelo pesquisador.

Duas formas de uso:

```
# linha de comando: prepara e, com --run, executa POA1 + Conservancy local
python -m poa.cli.import_realdata \
  --pred-dir ".../5_predicao_de_epitopo" \
  --world-dir ".../2_fastas_mundo" \
  --out      "./realdata_run" \
  --protein E --chikv-protein E1 --threshold 70 --run
```

Na GUI, a etapa 2 oferece o mesmo adaptador como caminho de upload, podendo adotar a referência reconstruída como `-f` caso ainda não tenha sido fornecida.

Resultado do teste ponta a ponta: POA1 + Conservancy local rodaram para CHIKV, DENV1, DENV2 e DENV4, com 13 a 16 epítopos por vírus e percentuais de conservação coerentes. DENV3 foi pulado por conter `X` na referência.

7. Correções

7.1 Parser do NetCTL 1.2 (correção científica, aprovada pelo usuário)

Bug **pré-existente** em `netctlAntigenEpitopes`, com duas causas somadas:

1. As linhas eram inseridas usando o número da linha do arquivo como índice não sequencial (`.loc[i]`), operação rejeitada pelo pandas ≥ 2 .
2. A lógica `if new_df_line[-1] != 'E': append('-')` estourava as 16 colunas nas linhas de epítipo reais, de modo que o critério de seleção `Identified_MHC_ligands == '<-E'` **nunca podia corresponder** — isto é, nenhum epítipo do NetCTL era selecionado.

Correção aplicada (índice sequencial; preenchimento apenas de linhas curtas até a contagem de colunas, preservando o marcador `<-E` nas linhas de epítipo; guarda para entrada vazia retornando as 7 colunas padrão). Como isso muda quais linhas são selecionadas, a alteração foi submetida e **aprovada explicitamente pelo usuário**. Coberta por testes de caminho feliz, ausência de dados e exclusão de linhas não-epítipo.

7.2 Robustez

- Criação automática dos diretórios de saída do POA1 (`-d`) e do POA2, evitando falha quando a pasta ainda não existe (sem impacto científico).
- Inclusão da raiz do repositório no `sys.path` para que `streamlit run poa/gui/app.py` resolva o pacote `poa` corretamente.
- Adequações ao pandas 3.0 encontradas durante a refatoração (`df.loc[índice_não_sequencial]` e `apply(axis=1)` sobre frames vazios agora levantam exceção; *Copy-on-Write* passou a ser padrão).

8. Testes

Suíte com **54 testes**, todos passando (`pytest` , configuração em `pytest.ini`):

Arquivo	Cobertura
tests/test_parsers.py	os seis parsers de formato de predição
tests/test_ranking.py	filtros e ranqueamento (lógica científica)
tests/test_pipeline.py	orquestração POA1/POA2
tests/test_topology.py	classificação de topologia de membrana
tests/test_conservancy_local.py	equivalência da conservação local com o formato IEDB
tests/test_clients.py , test_services.py	clientes de serviço, cache e fallback
tests/test_browser_client.py	automação de navegador (com mocks)
tests/test_gui_backend.py	orquestração da GUI e adaptador de dados reais
tests/test_viz.py	mapa 2D e geração do HTML do visualizador 3D
tests/test_realdata_import.py	adaptador dos arquivos originais da pesquisa

```
.venv/Scripts/python.exe -m pytest      # Windows
python -m pytest                        # Linux/WSL
```

9. Documentação adicionada

- **ARCHITECTURE.md** (442 linhas) — descreve o pipeline original exatamente como estava, marca cada ponto de intervenção manual, apresenta a investigação de disponibilidade de cada ferramenta externa em 2026 e detalha o plano de automação. É a referência para entender *por que* cada estratégia de automação foi escolhida.
- **TESTING_WSL.md** (282 linhas) — guia passo a passo para rodar as partes que dependem de Linux (EMBOSS `antigenic` , NetCTL standalone, pyTMHMM) em WSL2 + conda, incluindo a receita que efetivamente funciona para instalar o pyTMHMM (`numpy<2` + `--no-build-isolation`) e o uso de `conda-forge --override-channels` para evitar o bloqueio dos termos de uso da Anaconda.
- **README.md** — nova seção 0 documentando a camada de automação, a GUI, a instalação dos extras e o status por ferramenta.
- **requirements.txt** e **.gitignore** — criados.

10. Compatibilidade — o que não mudou

Esta é a garantia central da entrega:

- **A lógica científica de ranqueamento e seleção de epítomos é a mesma**, com a única exceção da correção do NetCTL descrita em 7.1, que foi aprovada explicitamente por corrigir um comportamento

comprovadamente errado.

- **A linha de comando é idêntica.** Os mesmos argumentos, os mesmos nomes de arquivo de saída:

```
bash python POA1_v1.0.py -f proteins.fasta -d results -b3 bepipred3.fasta -e y python POA2_v1.0.py -g True -t 70 -d conservancy_csvs -f proteins.fasta -rf 1
```

- **Toda automação é opcional.** Qualquer ferramenta pode continuar sendo usada manualmente, via upload do arquivo de resultado, exatamente como no fluxo original.
- **A licença MIT foi mantida.**

11. Limitações e pendências conhecidas

Item	Situação
Conservação local	Precisa ser validada contra uma exportação real do IEDB para o conjunto de dados em uso antes do uso em produção.
Automação de NetCTL / BepiPred-2.0	Experimental. Os seletores foram escritos contra as páginas da DTU e podem quebrar; o upload manual é o caminho confiável.
POA2 no Windows	Bloqueado — o pyTMHMM não instala nativamente. Use o WSL conforme <code>TESTING_WSL.md</code> .
EMBOSS, BepiPred-3.0, Playwright	Não instalados na máquina de desenvolvimento; esses caminhos não puderam ser verificados em execução real no Windows.
DENV3	Não processável enquanto o resíduo ambíguo <code>X</code> na referência não for resolvido a montante.
Visualização 3D	Depende de internet (3Dmol.js via CDN) e de numeração de resíduos do PDB compatível com as posições das predições.
Branch	<code>feature/viz-realdata</code> ainda não foi enviada ao repositório remoto.

12. Histórico de commits (`main` → `feature/viz-realdata`)

Commit	Descrição
<code>5b8bd7e</code>	docs: plano de arquitetura/automação, requirements e gitignore
<code>366c0b8</code>	refactor: extração do pipeline para o pacote <code>poa</code> (core/cli) com suíte pytest
<code>f908aa8</code>	feat(services): estrutura de cache/fallback manual + Conservancy Analysis local
<code>28798e5</code>	feat(services): clientes MHC-II (API IEDB), EMBOSS antigenic, pyTMHMM e BepiPred-3.0
<code>5618e9f</code>	feat(gui): aplicação Streamlit orquestrando o pipeline (fluxo de 6 etapas)
<code>25d6931</code>	feat(services): automação Playwright <i>best-effort</i> para NetCTL/BepiPred-2.0
<code>8b142d8</code>	docs(readme): camada de automação, GUI, instalação (EMBOSS/WSL) e CLI
<code>bf1e055</code>	fix(netctl): correção da seleção de epítomos (correção científica aprovada)
<code>424d620</code>	docs: guia passo a passo de testes em WSL (<code>TESTING_WSL.md</code>)
<code>24c049f</code>	docs: <code>conda-forge --override-channels</code> no guia WSL
<code>0a7434c</code>	docs: receita funcional de instalação do pyTMHMM em WSL
<code>3ab5555</code>	fix(pipeline): criação automática dos diretórios de saída de POA1/POA2
<code>1d441a0</code>	fix(gui): raiz do repositório no <code>sys.path</code> para <code>streamlit run</code>
<code>7729348</code>	feat(realdata): adaptador para os arquivos originais do fluxo manual
<code>9e3490d</code>	feat(gui): mapa 2D de epítomos, visualizador 3D e etapa de importação de dados reais

13. Como executar

```
# Ambiente
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # Linux/macOS
pip install -r requirements.txt

# Interface gráfica (fluxo completo de 7 etapas)
streamlit run poa/gui/app.py

# Linha de comando (inalterada)
python POA1_v1.0.py -f proteins.fasta -d results -b3 bepipred3.fasta -e y
python POA2_v1.0.py -g True -t 70 -d conservancy_csvs -f proteins.fasta -rf 1

# Dados originais da pesquisa
python -m poa.cli.import_realdata --pred-dir <5...> --world-dir <2...> \
    --out ./realdata_run --protein E --chikv-protein E1 --threshold 70 --run

# Testes
python -m pytest
```

Extras opcionais (não são pacotes pip ou são pesados — instale apenas se quiser aquela automação):

- **EMBOSS antigenic** (substitui o site PAP/IMED, fora do ar): `conda install -c bioconda emboss` (no Windows: WSL2 + conda, ou Docker).
- **BepiPred-3.0** local (PyTorch + ESM-2): `pip install bp3`.
- **Automação de navegador** para NetCTL 1.2 / BepiPred-2.0: `pip install playwright && playwright install chromium`.